

Пространственная фильтрация полутоновых изображений

1. Методы пространственной фильтрации полутоновых изображений

Если при изменении яркости полутонового изображения в точке с координатами (x, y) окрестность точки имеет произвольный вид, то процедура пространственной обработки будет в общем случае состоять из следующих действий, выполняемых для всех точек изображения:

1. Определить текущую точку (x, y) как центральную в определенной окрестности;
2. Выполнить заданную операцию с использованием лишь значений пикселей в окрестности вокруг центральной точки;
3. Заменить текущее значение в центральной точке значением результата операции, выполненной на предыдущем шаге.

При выполнении операции на втором шаге процедуры обработки используется матрица, размеры которой совпадают с размерами окрестности. Такую матрицу называют фильтром, маской, ядром, шаблоном или окном, а значения ее элементов – коэффициентами. Для описания самой операции принято использовать термины окрестностная обработка и пространственная фильтрация.

Различают линейную пространственную фильтрацию и нелинейную пространственную фильтрацию. В первом случае операции, совершаемые над пикселями окрестности, являются линейными. Во втором случае допускаются нелинейные операции. Свойство линейности определяется следующим образом:

$$T\left[\sum_{i=1}^n c_i r_i\right] = \sum_{i=1}^n c_i T[r_i], \quad (1)$$

где c_i – постоянные коэффициенты, не равные нулю; r_i – преобразуемые величины; T – оператор преобразования.

Операции, используемые при линейной пространственной фильтрации изображений, заключаются в суммировании результатов умножения значения каждого пикселя окрестности на соответствующий коэффициент маски.

Часто применяют квадратную окрестность $n \times n$ с нечетным n , например, 3×3 , 5×5 и т. д. При этом полагают, что центральный пиксел имеет координаты $(0, 0)$, т.е. коэффициенты маски значения координат меняются в интервале $\left[-\frac{n-1}{2}, \frac{n-1}{2}\right]$. Тогда линейную пространственную фильтрацию можно описать с помощью выражения:

$$g(x, y) = \sum_{i=-\frac{n-1}{2}}^{\frac{n-1}{2}} \sum_{j=-\frac{n-1}{2}}^{\frac{n-1}{2}} w(i, j) f(x + i, y + j), \quad (2)$$

где f – исходное изображение, g – обработанное изображение, w – маска размерами $n \times n$. Маску произвольного вида можно привести к квадратной форме, положив некоторые коэффициенты новой маски, равными нулю.

Процедура линейной пространственной фильтрации соответствует операции свертки:

$$g(x, y) = \sum_{i=N_1}^{N_2} \sum_{j=M_1}^{M_2} h(x - i, y - j) f(i, j) = \sum_{i=N_1}^{N_2} \sum_{j=M_1}^{M_2} h(i, j) f(x - i, y - j), \quad (3)$$

где h – импульсная характеристика фильтра (отклик фильтра на двумерную дискретную дельта-функцию). Поэтому линейную фильтрацию часто называют сверткой маски с изображением, а результат операции – откликом по маске. Если отличные от нуля значения функции h располагаются только в ограниченной области, то фильтр называется фильтром с конечной импульсной характеристикой или КИХ-фильтром, иначе – фильтром с бесконечной импульсной характеристикой или БИХ-фильтром.

В случае, когда интерес представляет только значение отклика по маске в точке (x, y) , а не схема реализации маски свертки, иногда используют следующее выражение:

$$s = \sum_{i=1}^N c_i r_i, \quad (4)$$

где s – отклик по маске, r_i – значения пикселей окрестности, представленной в виде одномерного массива, c_i – значения коэффициентов маски, представленной в виде одномерного массива, N – количество точек окрестности.

Одной из самых часто применяемых операций линейной пространственной фильтрации является сглаживание изображения, в результате которой вычисляется среднее значение яркости по окрестности. Соответствующий фильтр называют усредняющим, сглаживающим или низкочастотным фильтром.

Идея применения сглаживающих фильтров заключается в уменьшении резких изменений уровней яркости. Это может применяться для удаления шума на изображении, поскольку шум характеризуется скачками яркости. Еще одним применением является сглаживание ложных контуров на изображении, которые возникают при получении цифрового изображения из-за квантования с недостаточным числом уровней яркости. Также часто он применяется для сокращения информационной избыточности изображения при его сжатии и распознавании.

Таким образом, сглаживающие фильтры используются для подавления несущественных деталей на изображении, под которыми понимаются совокупности пикселей с размерами, малыми по сравнению с размерами маски фильтра.

Усредняющий фильтр, все коэффициенты маски которого одинаковы, называют однородным усредняющим фильтром. Недостаток таких фильтров заключается в возможном появлении ложного узора.

В общем случае коэффициенты маски могут быть различными, что позволяет ранжировать пиксели окрестности (присвоить им разные веса). Введение весов позволяет получить взвешенное среднее. Например, если присвоить центральному пикселу окрестности наибольший вес, а остальным пикселям – веса в зависимости от расстояния до центра, то можно уменьшить расфокусировку изображения при сглаживании. Для предотвращения смещения средней яркости сглаженного изображения относительно исходного сумма коэффициентов маски должна быть равна единице. Примеры масок сглаживающих фильтров:

$$W_1 = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, W_2 = \frac{1}{10} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}, W_3 = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}. \quad (5)$$

Маска W_3 , приведенная выше, задает низкочастотный гауссов фильтр с параметром $\sigma = 1$. Коэффициенты маски гауссова фильтра определяются преобразованием в дискретную форму непрерывной гауссовой функции:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left\{-\frac{x^2 + y^2}{2\sigma^2}\right\}, \quad (6)$$

где параметр x, y – расстояния до точки $(0, 0)$ по соответствующим координатным направлениям, а σ имеет смысл среднеквадратического отклонения нормального распределения вероятностей.

Дискретный гауссов фильтр имеет ошибку, связанную с конечностью размеров окна (непрерывная гауссова функция простирается до бесконечности в обоих направлениях). Ошибка уменьшается с увеличением размера окна. Это имеет значение при больших значениях σ .

В различных системах одним из этапов предварительной обработки изображений является повышение их резкости. Главная цель этой операции заключается в том, чтобы подчеркнуть мелкие детали изображения или улучшить те детали, которые оказались расфокусированы из-за ошибок или несовершенства условий и средств съемки.

Пространственный фильтр повышения резкости основан на применении следующего уравнения:

$$g = af - \hat{f}, \quad (7)$$

где f – исходное изображение, g – результирующее изображение, $\hat{f}$ – сглаженная версия исходного изображения, a – постоянный коэффициент, $a \geq 1$.

Для повышения резкости изображений часто применяют линейные пространственные фильтры, базирующиеся на использовании операторов первой и второй производных.

К операторам вычисления второй производной относится лапласиан, который является изотропным фильтром, т.е. фильтром, инвариантным к повороту (поворот изображения и последующее применение фильтра дает тот же результат, что и применение фильтра к исходному изображению с последующим

поворотом результата). В общем виде лапласиан двумерной функции $f(x, y)$ определяется следующим образом:

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}. \quad (8)$$

Для практического применения в обработке изображений лапласиан необходимо выразить в дискретной форме. Часто для этого используются следующие выражения:

$$\begin{aligned} \frac{\partial^2 f}{\partial x^2} &= f(x+1, y) + f(x-1, y) - 2f(x, y), \\ \frac{\partial^2 f}{\partial y^2} &= f(x, y+1) + f(x, y-1) - 2f(x, y), \\ \nabla^2 f(x, y) &= f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y). \end{aligned} \quad (9)$$

Приведенная форма лапласиана соответствует маске

$$W = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}. \quad (10)$$

Еще одна маска лапласиана, при вычислении которого используются диагональные пиксели, имеет следующий вид:

$$W = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}. \quad (11)$$

Применение лапласиана позволяет подчеркнуть разрывы уровней яркостей на изображении и подавить области со слабыми изменениями яркостей. Обобщенный алгоритм использования лапласиана для повышения резкости сводится к следующему:

$$g(x, y) = af(x, y) \pm \nabla^2 f(x, y), \quad (11)$$

где g, f – обработанное и исходное изображения, a – постоянный коэффициент, $a \geq 1$. В выражении (11) используется знак «плюс» при положительном центральном элементе маски лапласиана и знак «минус» при отрицательном центральном элементе маски лапласиана. Для повышения скорости обработки за счет выполнения только одного прохода можно использовать готовую маску фильтра повышения резкости, полученную на основе формулы (11).

В обработке изображений применяются также градиентные методы, основанные на использовании операторов первых производных. Градиент двумерной функции $f(x, y)$ – это вектор, ориентированный по направлению ее наиболее быстрого возрастания и имеющий длину, пропорциональную максимальному значению скорости это возрастания. Компоненты вектора представляют собой частные производные функции f по координатам x и y :

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}. \quad (12)$$

Фаза градиента характеризует направление возрастания анализируемой функции, а модуль – скорость возрастания. Эти характеристики вычисляются следующим образом:

фаза:

$$\arg \nabla f(x, y) = \arctan \left(\left[\frac{\partial f / \partial y}{\partial f / \partial x} \right] \right), \quad (13)$$

модуль:

$$|\nabla f(x, y)| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}. \quad (14)$$

Для получения градиента требуется выполнить относительно большой объем вычислений. Поэтому часто применяют следующее упрощение:

$$|\nabla f(x, y)| \approx \left| \frac{\partial f}{\partial x} \right| + \left| \frac{\partial f}{\partial y} \right|. \quad (15)$$

Простейшими приближениями первых производных являются:

$$\begin{aligned} \frac{\partial f}{\partial x} &= f(x+1, y) - f(x, y), \\ \frac{\partial f}{\partial y} &= f(x, y+1) - f(x, y). \end{aligned} \quad (16)$$

Данные выражения соответствуют маскам:

$$W_x = \begin{bmatrix} - & 1 \\ 1 & \end{bmatrix}, \quad W_y = \begin{bmatrix} - & 1 & 1 \end{bmatrix}. \quad (17)$$

Следующий вариант называется оператором Робертса:

$$\begin{aligned}\frac{\partial f}{\partial x} &= f(x+1, y) - f(x, y+1), \\ \frac{\partial f}{\partial y} &= f(x, y) - f(x+1, y+1).\end{aligned}\quad (18)$$

Маски оператора Робертса имеют вид

$$W_x = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}, \quad W_y = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}. \quad (19)$$

Масками (17) и (19), которые имеют размеры 1×2 и 2×2 , пользоваться не очень удобно. Поэтому для расчета были предложены маски с размерами 3×3 , основанные, например, на уравнениях:

$$\begin{aligned}\frac{\partial f}{\partial x} &= [f(x+1, y-1) + 2f(x+1, y) + f(x+1, y+1)] - \\ &\quad - [f(x-1, y-1) + 2f(x-1, y) + f(x-1, y+1)], \\ \frac{\partial f}{\partial y} &= [f(x-1, y+1) + 2f(x, y+1) + f(x+1, y+1)] - \\ &\quad - [f(x-1, y-1) + 2f(x, y-1) + f(x+1, y-1)].\end{aligned}\quad (20)$$

Соответствующие маски называют операторами Собела и выглядят следующим образом:

$$W_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad W_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}. \quad (21)$$

Также часто используются операторы Превитта:

$$W_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \quad W_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}. \quad (22)$$

Повышение резкости с помощью операторов градиента выполняется следующим образом:

$$g(x, y) = af(x, y) + \nabla f(x, y), \quad (23)$$

где g, f – обработанное и исходное изображения, a – постоянный коэффициент, $a \geq 1$. Для получения ∇f используется процедура:

1. Обработка изображения f фильтрами производных по x и y , результатом которой являются изображения f_x и f_y ;

2. Формирование градиентного изображения:

$$\nabla f = |f_x| + |f_y|; \quad (24)$$

Таким образом, фильтры производных (лапласиан и градиент) являются высокочастотными фильтрами, позволяющими выделить перепады яркостей, что имеет важное значение не только для повышения резкости, но и для выделения контуров, например, при распознавании изображений.

Однако при использовании этих фильтров усиливаются шумы на изображениях. Снизить данный эффект можно с помощью LoG-оператора, действие которого аналогично последовательной обработке изображения сначала гауссианом, а затем лапласианом. Маска LoG-фильтра получается дискретизацией и нормировкой следующей непрерывной функции:

$$LoG(x, y, \sigma) = \frac{1}{2\pi\sigma^2} \left(\frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} \right) \exp \left\{ -\frac{x^2 + y^2}{2\sigma^2} \right\}. \quad (25)$$

Например, маска LoG-фильтра с $\sigma = 0.9$ может иметь вид

$$W = -\frac{1}{8} \begin{bmatrix} 0 & 2 & 2 & 2 & 0 \\ 2 & 1 & -4 & 1 & 2 \\ 2 & -4 & -20 & -4 & 2 \\ 2 & 1 & -4 & 1 & 2 \\ 0 & 2 & 2 & 2 & 0 \end{bmatrix}. \quad (26)$$

Аппроксимацией LoG-фильтра служит DoG-фильтр, соответствует разности двух гауссовых функций с различной дисперсией:

$$DoG(x, y, \sigma) = \frac{1}{2\pi} \left[\frac{1}{\sigma_1^2} \exp \left\{ -\frac{x^2 + y^2}{2\sigma_1^2} \right\} - \frac{1}{\sigma_2^2} \exp \left\{ -\frac{x^2 + y^2}{2\sigma_2^2} \right\} \right], \quad (27)$$

где $\sigma_1 > \sigma_2$. Применение фильтра DoG позволяет сократить время вычислений.

Рассмотренные до сих пор фильтры относятся к классу линейных фильтров. На практике также часто используются нелинейные фильтры. Простыми и в то же время эффективными нелинейными пространственными фильтрами являются ранговые фильтры, основанные на порядковых статистиках. Отклик такого фильтра определяется предварительным упорядочиванием значений пикселей, соответствующих маске фильтра, и последующим выбором значения, находящегося на определенной позиции упорядоченной последовательности. Сама фильтрация сводится к замещению исходного значения пиксела в центре маски на полученное значение отклика фильтра. При этом для увеличения веса

определенного пиксела окрестности его можно вставлять в упорядоченную последовательность заданное число раз.

Наиболее известен медианный фильтр, который заменяет значение центрального пиксела на значение медианы распределения яркостей пикселей в окрестности (медиана – это середина упорядоченной последовательности), включая и исходный. Они используются для сглаживания изображений и подавления шума. По сравнению с обработкой линейными сглаживающими фильтрами эффект расфокусировки в этом случае проявляется в меньшей степени.

К фильтрам, основанным на порядковых статистиках, относятся также фильтры максимума и минимума, откликами которых являются максимальное и минимальное значения соответственно.

Эффективными нелинейными фильтрами являются билатеральные фильтры, которые применяются в различных задачах обработки изображений. Результатом работы билатерального фильтра является вычисление яркости каждого пиксела, как взвешенного среднего яркостей пикселей его окрестности. При этом учитывается не только расстояние между пикселями, но и степень схожести их яркостей. Один из вариантов билатеральной фильтрация выполняется следующим образом:

$$g(x, y) = \frac{\sum_{i=-\frac{n-1}{2}}^{\frac{n-1}{2}} \sum_{j=-\frac{n-1}{2}}^{\frac{n-1}{2}} w(i, j) f(x+i, y+j)}{\sum_{i=-\frac{n-1}{2}}^{\frac{n-1}{2}} \sum_{j=-\frac{n-1}{2}}^{\frac{n-1}{2}} w(i, j)}, \quad (28)$$

$$w(i, j) = K \exp\left(-\frac{i^2 + j^2}{2\sigma_1^2}\right) \exp\left(-\frac{(f(x+i, y+j) - f(x, y))^2}{2\sigma_2^2}\right),$$

где K – нормирующая константа, σ_1 – пространственный параметр, σ_2 – яркостный (ранговый) параметр. Применение подобного фильтра позволяет сохранять четкие границы элементов изображения.

Выше, в основном, были рассмотрены низкочастотные (для сглаживания) и высокочастотные (для выделения границ) пространственные фильтры. Их

композиция (последовательное применение) позволяет получить еще один вид фильтров – полосовые фильтры, которые используются для выделения определенных деталей изображения.

В заключение отметим особенности обработки пикселей, расположенных на границе изображения. Здесь, как правило, используется один из следующих способов:

- 1) фильтрация для граничных пикселей не выполняется;
- 2) пиксели, которые находятся за границами изображения, не используются при суммировании, а их вес равномерно распределяется среди других пикселей окрестности;
- 3) пиксели за границами изображения доопределяются при помощи экстраполяции или зеркального отражения.

2. Пространственная фильтрация в OpenCV

Для реализации рассмотренных методов пространственной фильтрации в OpenCV предусмотрены следующие функции:

```
void cvFilter2D(const CvArr* src,
               CvArr* dst,
               const CvMat* kernel,
               CvPoint anchor=cvPoint(-1,-1));

void cvSmooth(const CvArr* src,
              CvArr* dst,
              int smoothtype = CV_GAUSSIAN,
              int size1 = 3,
              int size2 = 0,
              double sigma1 = 0,
              double sigma2 = 0);

void cvSobel(const CvArr* src,
             CvArr* dst,
             int xorder,
             int yorder,
```

```

        int aperture_size = 3);
void cvLaplace(const CvArr* src,
               CvArr* dst,
               int aperture_size = 3);

```

Функция `cvFilter2D` определяет пространственную обработку изображения `src` по маске `kernel`. Результат обработки сохраняется как изображение `dst`. Размещение маски относительно оклика фильтра определяется параметром `anchor`. Если задано значение параметра `anchor`, равное `cvPoint(-1,-1)`, то пиксел-отклик располагается по центру маски. применение собственной маски для обработки изображения. Для создания маски можно воспользоваться функцией `cvCreateMat`:

```
CvMat* cvCreateMat(int rows, int cols, int type);
```

Ее параметрами являются `rows` – число строк, `cols` – число столбцов и `type` – тип элементов матрицы, который задается как константа

```
CV_<bit_depth><S|U|F>C<number_of_channels>
```

где `<bit_depth>` – число битов на канал, `S` – целое со знаком, `U` – целое без знака, `F` – вещественное, `<number_of_channels>` – число каналов для представления элемента. Заполнить созданную матрицу-маску можно, например, с помощью функции `cvSetReal2D`.

Функция `cvSmooth` предназначена для сглаживания изображений. Она имеет следующие аргументы:

`src` – исходное изображение,
`dst` – результирующее изображение,
`smoothtype` – тип сглаживания:

`CV_BLUR_NO_SCALE` – суммирование без масштабирования,

`CV_BLUR` – суммирование с последующим делением результата на значение `size1 × size2`,

`CV_GAUSSIAN` – свёртка изображения с гауссовым ядром размерами `size1 × size2`,

CV_MEDIAN – медианное сглаживание (поиск среднего значения в окрестности $size1 \times size2$),

CV_BILATERAL – билатеральная фильтрация (сглаживающая фильтрация, при которой значение интенсивности пиксела определяется средневзвешенным значением интенсивностей соседних пикселей, причём веса зависят не только от пространственного расстояния, но и от расстояния в цветовом пространстве) по окрестности $size1 \times size1$ (если $size1 = 0$, то размеры области определяются выражением $cvRound(sigma2*1.5)*2+1$, где $cvRound$ – функция округления) с цветовым стандартным отклонением $sigma1$ и пространственным стандартным отклонением $sigma2$,

$size1$ – размер области сглаживания по ширине (должен быть нечетным значением: 1, 3, 5, ...),

$size2$ – размер области сглаживания по высоте (этот аргумент игнорируется при медианной и билатеральной фильтрации, в остальных случаях должен быть нечетным значением либо нулем, нулевое значение задает высоту, равную значению $size1$),

$sigma1$ – цветовое стандартное отклонение, которое используется при гауссовой и билатеральной фильтрации, если этот параметр равен нулю, то стандартное отклонение определяется по формуле:

$$\sigma = 0.3(n/2 - 1) + 0.8, \quad (29)$$

где n – размер по ширине либо по высоте,

$sigma2$ – пространственное стандартное отклонение.

Функция `cvSobel` вычисляет первую, вторую, третью производные, в том числе и смешанные, на основе оператора Собела или Щарра. Аргументы функции:

`src` – исходное изображение,

`dst` – результирующее изображение,

`xorder` – порядок производной по x ,

`yorder` – порядок производной по y ,

`aperture_size` – размер ядра оператора производной (1,3,5,7 – оператор Собела, –1 – оператор Щарра).

Оператор Щарра имеет маски:

$$W_x = \begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}, W_y = \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}. \quad (30)$$

Функция `cvLaplace` позволяет получить лапласиан изображения. Ее аргументы аналогичны таким же аргументам функции `cvSobel`.

Для корректного выполнения вычисления лапласиана и производных лучше использовать для результирующего изображений 16/32-разрядное целочисленное либо 32/64-разрядное вещественное представление пикселей с последующим приведением к 8-разрядному беззнаковому представлению с помощью `cvConvert` с аргументами `src` – исходное изображение, `dst` – результирующее изображение. Кроме того, может быть полезно использование `cvAbs`, что позволяет получить результирующее изображение `dst`, пиксели которого имеют абсолютные значения пикселей исходного изображения `src`.

Листинг 1 содержит программу, иллюстрирующую методы сглаживания. Пример выполнения этой программы показан на рис. 1. При создании и исполнении проекта используются файлы включения `cv.h` и `highgui.h`, файлы библиотек `opencv_core249d.lib`, `opencv_highgui249d.lib` и `opencv_imgproc249d.lib`, исполняемые файлы `opencv_core249d.dll`, `opencv_highgui249d.dll` и `opencv_imgproc249d.dll`.

3. Задание

1. Разработать функцию усредняющего сглаживания изображения.
2. Разработать функцию гауссовой фильтрации изображения.
3. Разработать функцию медианной фильтрации изображения.
4. Разработать функцию вычисления градиента методом Робертса.
5. Разработать функцию вычисления градиента методом Собела.
6. Разработать функцию вычисления градиента методом Превитта.

Сравнить результаты с результатами работы функций OpenCV.

ЛИСТИНГ 1.

```
#include <opencv/cv.h>
#include <opencv/highgui.h>
int main(int argc, char ** argv)
{
    IplImage * source;
    IplImage * blur;
    IplImage * gaussian;
    IplImage * median;
    IplImage * bilateral;

    source = cvLoadImage(argv[1], CV_LOAD_IMAGE_GRAYSCALE);
    blur = cvCreateImage(cvSize(source->width, source->height),
                        IPL_DEPTH_8U, 1);

    gaussian = cvCreateImage(
        cvSize(source->width, source->height), IPL_DEPTH_8U, 1);
    median = cvCreateImage(cvSize(source->width, source->height),
                        IPL_DEPTH_8U, 1);
    bilateral = cvCreateImage(
        cvSize(source->width, source->height), IPL_DEPTH_8U, 1);

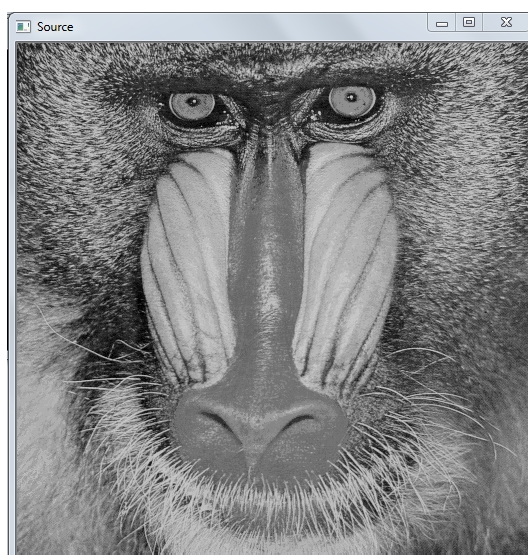
    cvSmooth(source, blur, CV_BLUR, 7);
    cvSmooth(source, gaussian, CV_GAUSSIAN, 7);
    cvSmooth(source, median, CV_MEDIAN, 7);
    cvSmooth(source, bilateral, CV_BILATERAL, 7);

    cvNamedWindow("Source", CV_WINDOW_AUTOSIZE);
    cvShowImage("Source", source);
    cvNamedWindow("Blur", CV_WINDOW_AUTOSIZE);
    cvShowImage("Blur", blur);
    cvNamedWindow("Gaussian", CV_WINDOW_AUTOSIZE);
    cvShowImage("Gaussian", gaussian);
    cvNamedWindow("Median", CV_WINDOW_AUTOSIZE);
    cvShowImage("Median", median);
    cvNamedWindow("Bilateral", CV_WINDOW_AUTOSIZE);
    cvShowImage("Bilateral", bilateral);

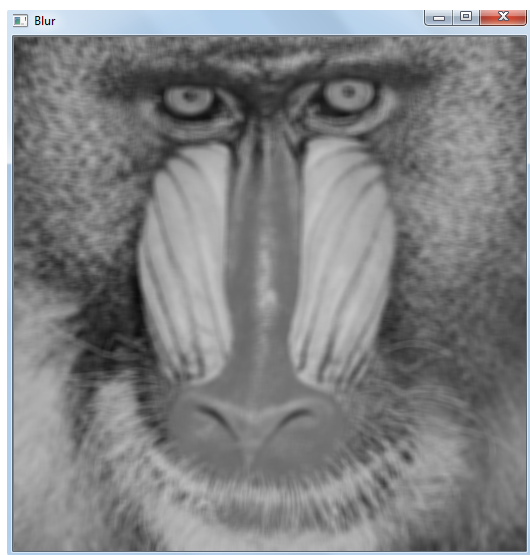
    cvWaitKey(0);

    cvReleaseImage(&source);
    cvReleaseImage(&blur);
    cvReleaseImage(&gaussian);
    cvReleaseImage(&median);
    cvReleaseImage(&bilateral);
    cvDestroyAllWindows();

    return 0;
}
```



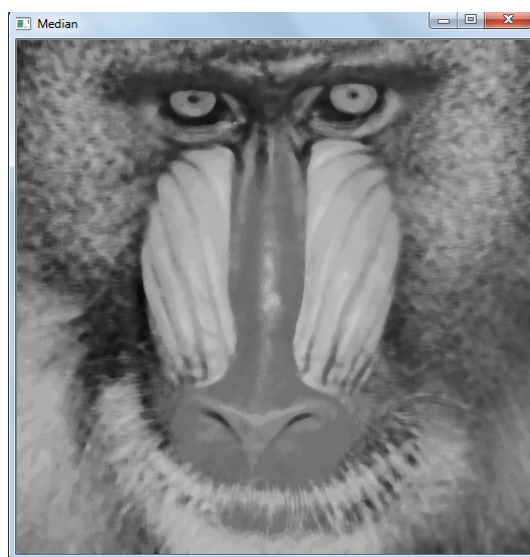
а) Исходное изображение



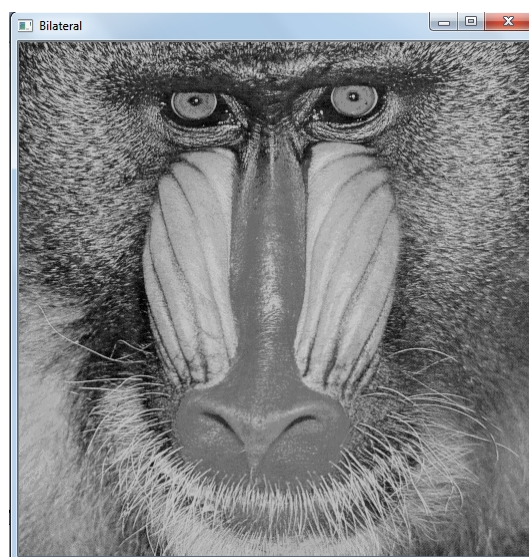
б) Усредняющий фильтр



в) Гауссов фильтр



г) Медианный фильтр



д) Билатеральный фильтр

Рис. 1. Пространственная фильтрация